

# Altrium Recruitment

Recruitment & hiring tracker — Scenario 1, built for Altrium.

Complete handover: what it is, how it was built, how the database works, how to run it, how to change it, and how it is deployed.

|                 |                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| Live site       | <a href="https://hiretrack1.onrender.com">https://hiretrack1.onrender.com</a>                                 |
| Source code     | <a href="https://github.com/hiretracktest-hue/hiretrack1">https://github.com/hiretracktest-hue/hiretrack1</a> |
| Database        | PostgreSQL on Supabase — project mxahawsqubspavtgrgia                                                         |
| CV storage      | Supabase Storage, private bucket 'candidate-cvs'                                                              |
| Email           | Resend API                                                                                                    |
| Automated tests | 77, all passing                                                                                               |
| Group           | Isuru, Fazl, Thariq and Ahmed                                                                                 |

## The four sign-ins

| Role           | Person                | Email                                                                          | Password   |
|----------------|-----------------------|--------------------------------------------------------------------------------|------------|
| HR Recruiter   | Nimali Wijesinghe     | <a href="mailto:hr@hiretrack.test">hr@hiretrack.test</a>                       | hr12345    |
| Hiring Manager | Chathura Rajapaksha   | <a href="mailto:hiringmanager@hiretrack.test">hiringmanager@hiretrack.test</a> | hm12345    |
| Interviewer    | Sanduni Ekanayake     | <a href="mailto:int@hiretrack.test">int@hiretrack.test</a>                     | int12345   |
| Management     | Mahesh<br>Gunawardena | <a href="mailto:manag@hiretrack.test">manag@hiretrack.test</a>                 | manag12345 |

The domain is **.test** — not **.com**. Candidates never sign in; there is no public sign-up at all.

# 1. What the system does

Hiring normally runs across inboxes, spreadsheets and chat. CVs arrive by email, interview feedback is scattered, and nobody has a live view of where a candidate is. This replaces that with one internal system.

HR opens a position, sets its interview stages, adds candidates and uploads their CVs. Interviewers are asked to confirm the interviews they are booked for, then leave feedback on a fixed form. The hiring manager compares candidates side by side and records the decision. Management watches the numbers and exports reports.

## What makes it more than a list

- **Stages are set per position.** They live in their own table with an order, so Junior Software Engineer can run Applied → Screening → Technical → Final → Offer while QA Engineer runs a different set entirely.
- **Nobody advances on missing information.** A candidate cannot move to the next stage until the current stage's feedback is in. The first stage is exempt — nobody has interviewed someone who was only just added.
- **Feedback is comparable.** Everyone fills the same form: score out of 5, an advance/hold/reject recommendation, strengths, concerns. A database constraint allows one score per person per stage, so nobody can weight the result by writing twice.
- **Interviewers answer, they are not just told.** A booking starts PENDING and the interviewer accepts or declines — from inside the app, or from a button in their email. A booking nobody answered is what quietly derails hiring, so it is tracked rather than assumed.
- **CV screening scales.** One advert can return hundreds of CVs, so each is read once and banded High / Medium / Low. The list then filters by band.

## 2. What it is built with

| Layer        | Technology                                               | Why                                                              |
|--------------|----------------------------------------------------------|------------------------------------------------------------------|
| Front end    | React 18 (JSX), React Router, plain CSS, built with Vite | No UI framework, so every style is ours and can be explained     |
| Back end     | Node.js + Express, REST API                              | One language across the stack                                    |
| Database     | PostgreSQL on Supabase, accessed with pg (node-postgres) | Real constraints and types, not a file                           |
| File storage | Supabase Storage, private bucket                         | CVs survive a redeploy and are not sitting in the project folder |
| Email        | Resend HTTP API, with SMTP as an alternative             | An HTTP call works where outbound SMTP ports are blocked         |
| Auth         | Email + password (bcrypt), JWT in an httpOnly cookie     | Page JavaScript cannot read the token, so XSS cannot steal it    |
| Tests        | Node's built-in test runner — 77 API tests               | No extra dependency to justify                                   |
| Hosting      | Render (free web service)                                | Runs Node; GitHub Pages cannot                                   |

## Visual design

The palette and typeface are taken from the client's own site, altrium.io: amber #fbb401 as the accent, #1e2228 headings, #60697b body text, #f6f7f9 page, set in Cabin, with pill buttons at weight 700.

**The amber never carries white text.** White on #fbb401 measures 1.8:1, far below the 4.5:1 WCAG AA requires — close to unreadable. Altrium put black on the amber themselves, and so do we: #1e2228 on #fbb401 gives 8.9:1. Where the brand has to appear as text rather than a fill, #8a6300 is used, which reaches 5.4:1 on white.

### 3. How the database works

Eight tables on PostgreSQL, defined in one file: `database/schema.sql`. Nothing is created by hand in the Supabase dashboard — the file is the single source of truth, so the schema can be rebuilt from scratch at any time.

```
users ■■■< jobs ■■■< job_stages
■■ ■
■■■■■< candidates ■■■< interviews ■■■< notifications
■■ ■
■■■■■< feedback
■■■■■< password_resets
```

| Table           | Holds                                                                       |
|-----------------|-----------------------------------------------------------------------------|
| users           | Everyone who signs in: HR, hiring managers, interviewers, management        |
| jobs            | Open positions, with description, department, location, salary range        |
| job_stages      | The interview stages for one position, in order                             |
| candidates      | People being considered. Their CV, band, stage, outcome, invitation details |
| interviews      | Bookings: when, where, who interviews, and their accept/decline answer      |
| feedback        | One score per person per stage: rating, recommendation, strengths, concerns |
| notifications   | In-app notifications and the candidate email outbox                         |
| password_resets | Single-use tokens for the forgotten-password flow                           |

#### What the database enforces for itself

- **Foreign keys with ON DELETE CASCADE** — deleting a position cleans up its stages, candidates, interviews and feedback. No orphan rows.
- **Eight ENUM types** for role, status, outcome, cv\_band, employment\_type, feedback recommendation, notification channel and interview response. An invalid value cannot be written at all.
- **A CHECK constraint** keeps a feedback rating between 1 and 5.
- **UNIQUE (candidate\_id, stage, author\_id)** on feedback — this is what makes the comparison fair.
- **A trigger** maintains updated\_at, so no query can forget to set it.
- **citext** makes email comparison case-insensitive, so Maya@gmail.com and maya@gmail.com are one person.
- **Every query is parameterised** — values never reach SQL as text, so SQL injection is not possible.

Report figures use separate subqueries rather than one large join. Joining candidates to feedback would count a candidate once per review and silently inflate every total — there is a test that catches exactly that.

## 4. Running it, and changing the data

### First time on a new machine

```
npm run setup # install everything, create the tables, add demo data
npm run dev # React on :5173, API on :4000
```

### Every command

| Command                       | What it does                                        |
|-------------------------------|-----------------------------------------------------|
| npm run dev                   | Run the API and the React dev server together       |
| npm start                     | Run the API, serving the built React app too        |
| npm run build                 | Build the React app into client/dist                |
| npm test                      | Run the 77 automated API tests                      |
| npm run db:migrate            | Create the tables from database/schema.sql          |
| npm run db:migrate -- --force | Same, but drops and rebuilds even with data present |
| npm run seed                  | Add any missing demo data (safe to re-run)          |
| npm run seed:reset            | Empty every table, then seed from scratch           |

### Removing things from the database

Four ways, from gentlest to most destructive.

**1. Through the app.** The normal way. Delete a candidate from their own page, or a position from the positions list. Only HR can, and cascade rules clean up everything attached — a deleted candidate takes their interviews, feedback and notifications with them, and their CV is removed from the bucket.

**2. Reset the demo data.** Empties every table and re-seeds it. This is the usual fix after testing has left a mess:

```
npm run seed:reset
```

**3. Rebuild the schema completely.** Drops every table and type, recreates them from schema.sql, then re-seeds. Use this after changing the schema:

```
npm run db:migrate -- --force
npm run seed
```

**4. By hand, in Supabase.** Dashboard → SQL Editor. Useful for removing one specific thing:

```
-- one candidate, by email
DELETE FROM candidates WHERE email = 'someone@example.com';

-- every candidate on one position
DELETE FROM candidates WHERE job_id = 3;

-- clear the notification outbox but keep everything else
DELETE FROM notifications;

-- see what is there first, before deleting anything
SELECT id, full_name, email, current_stage FROM candidates ORDER BY id;
```

Always run the SELECT first. DELETE without a WHERE clause empties the whole table, and there is no undo. The Table Editor is the safer route if you are unsure — it deletes one row at a time and asks first.

## Changing the demo people

The seeded staff and candidates are defined at the top of `database/seed.js` as plain lists. Edit the names, emails, roles or passwords there, then run `npm run seed:reset`.

## 5. Who can do what

Four roles with genuinely different permissions, defined in one place (`server/config.js`) and enforced by the API on every request. Hiding a button is convenience only — the server checks the same rule again, and there are tests that sign in as each role and confirm the refusals.

| Action                                  | HR  | Hiring Mgr | Interviewer | Mgmt |
|-----------------------------------------|-----|------------|-------------|------|
| Open / edit / close / delete a position | Yes | —          | —           | —    |
| See every candidate                     | Yes | Yes        | Yes         | Yes  |
| Add a candidate                         | Yes | —          | —           | —    |
| Upload a CV                             | Yes | —          | —           | —    |
| Download a CV                           | Yes | Yes        | Yes         | Yes  |
| Band a CV High / Medium / Low           | Yes | Yes        | —           | —    |
| Move a candidate to the next stage      | Yes | Yes        | —           | —    |
| Record hired / rejected / on hold       | Yes | Yes        | —           | —    |
| Leave interview feedback                | Yes | Yes        | Yes         | —    |
| Accept or decline an interview          | —   | —          | Yes         | —    |
| Schedule an interview                   | Yes | Yes        | —           | —    |
| Candidate email outbox                  | Yes | Yes        | —           | —    |
| View reports                            | Yes | Yes        | —           | Yes  |
| Export CSV                              | Yes | —          | —           | Yes  |
| Create accounts, set roles              | Yes | —          | —           | —    |

The Team page renders this table live from the API, so the documentation cannot drift away from the rules. There is no public sign-up: HR creates every account, and the last HR account cannot demote or deactivate itself.

### Security decisions worth naming

- Passwords hashed with **bcrypt**; the hash never leaves the server.
- Session in an **httpOnly** cookie, so page JavaScript — and therefore XSS — cannot read the token. `SameSite=Lax`, `Secure` in production.
- Sign-in and forgot-password give the **same answer** whether or not the email exists, so neither form can be used to harvest addresses.
- CVs are stored under a **generated random name** in a **private** bucket and handed out as signed URLs that expire in 60 seconds. The app refuses to use the bucket at all if it is public.
- **Any file type is accepted** for a CV, which is only safe because a CV is always returned as an attachment with `nosniff` and a locked-down CSP. An `.html` CV rendered in the page would be XSS; a test uploads a file containing a script tag and checks it comes back as a download.
- The **invitation link** in an interview email is a signed token scoped to one interview and one interviewer, expiring after 30 days. It cannot be used as a login, a login cannot be used as an invitation, and it exposes nothing beyond that one booking.

**Known limits, honestly stated.** Before this handled real candidate data it would also need: rate limiting on the sign-in route, virus scanning of uploads, and an audit log. The demo passwords in this document are fine for a demonstration and are not fine in front of real personal data.

## 6. How email works

Every message the system decides to send is **written to the database first**, in the notifications table. That row is the record of what was supposed to go out, and it does not depend on a mail server being reachable. Sending is a second step on top of it.

| Provider | When it is used                                                                                                                                    |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Resend   | RESEND_API_KEY and RESEND_FROM_EMAIL set. An HTTP API, so it works on networks that block outbound SMTP ports. Preferred when both are configured. |
| SMTP     | SMTP_HOST / SMTP_USER / SMTP_PASS set. Any mailbox, including Gmail with an App Password.                                                          |
| Neither  | Messages wait in the Outbox for HR to send by hand — exactly how the project worked before mail was wired up.                                      |

### What gets sent, and to whom

| Event               | Goes to     | Says                                                                                                                      |
|---------------------|-------------|---------------------------------------------------------------------------------------------------------------------------|
| HR adds a candidate | Candidate   | "Altrium has invited you to apply" — the role, department, location, any link and time HR attached, and what happens next |
| Interview booked    | Interviewer | "Can you interview X?" with Accept and Decline buttons                                                                    |
| Interviewer accepts | Candidate   | Interview confirmed, with date and interviewer                                                                            |
| Hired or rejected   | Candidate   | Offer letter or regret letter                                                                                             |

### Notifications inside the app are per role

One event produces a different message for each person who needs it, and nothing for those who do not:

An interviewer accepts a booking:

Interviewer "You accepted the interview with X"  
HR "Sanduni Ekanayake accepted the interview with X"  
Hiring Manager "Interview confirmed - X"  
Management (nothing - they watch hires, not calendars)

A **decline** goes only to HR, worded as an action and carrying the reason — they are the one who has to rebook, it is not the manager's problem yet, and the candidate has not been told anything. Nobody is ever notified about their own action, because being told what you just did is noise.

**Nothing is ever marked sent unless a provider accepted it.** When a send is refused, the provider's own reason is stored on the row and shown to HR in full, with a Send now button to retry. A refused email that merely looks unsent is indistinguishable from one nobody tried to send.

### Current limitation

The Resend account has no verified domain, so it delivers only to the address that owns the account, hiretracktest@gmail.com. Every other recipient is refused with a 403 and the reason is recorded. To reach any address, either verify a domain at resend.com/domains, or switch to Gmail SMTP with an App Password, which has no such restriction.

## 7. How it is deployed

**GitHub Pages cannot host this.** Pages serves static files only, so it cannot run Node, the API, the database connection, sign-in, uploads or email. Pointing Pages at the repository renders README.md as a web page — which is not the application.

One service hosts everything instead: `npm start` boots Express, which also serves the built React app. The database stays on Supabase, so nothing has to move.

### Render setup

| Setting           | Value                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------------|
| Service type      | Web Service (not Static Site)                                                                          |
| Language          | Node                                                                                                   |
| Region            | Singapore                                                                                              |
| Build command     | <code>npm install &amp;&amp; npm --prefix client install --include=dev &amp;&amp; npm run build</code> |
| Start command     | <code>npm start</code>                                                                                 |
| Health check path | <code>/api/health</code>                                                                               |
| Plan              | Free                                                                                                   |

**--include=dev matters.** Render sets `NODE_ENV=production`, which makes npm skip devDependencies — and vite, the thing that builds the React app, is a devDependency. Without it the build fails with "vite: not found".

### Environment variables

| Key                                    | Purpose                                                                                                                                                     |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>NODE_ENV</code>                  | production — turns on secure cookies and the secret check                                                                                                   |
| <code>JWT_SECRET</code>                | Signs every session cookie. The app refuses to start in production without a real one. Changing it signs everybody out.                                     |
| <code>DATABASE_URL</code>              | Supabase connection string. Must use the pooler host (...pooler.supabase.com:6543); the direct db.<ref>.supabase.co host is IPv6-only and will not connect. |
| <code>CLIENT_URL</code>                | The service's own https address. The Accept/Decline links in interview emails are built from it, so if it is wrong they point at localhost.                 |
| <code>SUPABASE_URL</code>              | Project URL, for CV storage                                                                                                                                 |
| <code>SUPABASE_SERVICE_ROLE_KEY</code> | Secret key for Storage. Bypasses row-level security, so it is read on the server only and never reaches the browser.                                        |
| <code>SUPABASE_CV_BUCKET</code>        | candidate-cvs                                                                                                                                               |
| <code>RESEND_API_KEY</code>            | Email sending. Optional.                                                                                                                                    |
| <code>RESEND_FROM_EMAIL</code>         | The From address on outgoing mail                                                                                                                           |

**Secrets never go in the repository.** Real values live in `.env`, which is in `.gitignore` and never committed. `.env.example` is the committed template and every value in it is blank — anything typed to the right of an `=` in that file becomes public the moment it is pushed.

### Things to expect



- The free plan sleeps after 15 minutes idle, so the first visit after a quiet spell takes about 50 seconds to wake. Open it a minute before a demonstration.
- The container's own disk is wiped on every deploy, which is why CVs go to Supabase Storage rather than a folder on the server.
- A health check at `/api/health` returns `ok:true` only when the app is running *and* connected to the database — not merely when the process started.

## 8. Testing

```
npm test
```

```
→ 77 tests, 77 passing
```

The tests run against the real PostgreSQL database but inside their own throw-away schema, created at the start and dropped at the end, so the real tables are never touched. They are grouped by the questions the brief asks: per-position stages, the feedback gate before advancing, fair comparison, interview notifications, the four roles, CV screening, and the reports.

### Two things the test setup does deliberately

- **The suite runs with no mail provider and no Storage bucket**, whatever is in `.env`. Without that, `npm test` would fire real messages at real addresses through a real account and write throw-away CVs into the production bucket.
- **Tests use the session pooler, not the transaction pooler**. In transaction mode the pooler hands the same backend to different clients, so the test's `SET search_path` leaked out of the run and into the live app — which is exactly what happened the first time these tests were run against Supabase.

### Some of what is covered

- Each of the four roles is signed in and refused the things it must not do.
- A candidate cannot advance while feedback for the current stage is missing.
- The reports add up and do not double-count.
- A CV containing a script tag comes back as a download, never rendered.
- A forged, expired or reassigned invitation link is refused; a sign-in token cannot be used as an invitation.
- Accepting an interview reaches HR and the hiring manager but not management; declining reaches HR only.
- The database refuses an invalid role, an out-of-range rating, and cascades a deletion correctly.

## 9. Where everything lives

```
our web/  
database/  
  schema.sql the whole schema - 8 tables, 8 types, a trigger  
  migrate.js creates the tables from schema.sql  
  seed.js the demo people, positions and candidates  
  index.js connection pool and query helpers  
server/  
  index.js starts the server  
  app.js builds the Express app, serves the built React app  
  config.js settings, the four roles, the permission map  
  auth.js password hashing, JWT, cookies, reset tokens  
  storage.js CVs: Supabase bucket, or local disk as a fallback  
  mail.js sending, and the signed invitation links  
  mail-templates.js the HTML of the emails we actually send  
  notify.js who hears about what, per role  
  validate.js input checks, including URL safety  
  routes/ one file per area of the API  
  tests/ 77 automated tests  
client/  
  src/pages/ one file per screen  
  src/styles.css the whole stylesheet  
  render.yaml deployment configuration
```

```
.env real secrets - never committed  
.env.example blank template - committed
```

---

Altrium Recruitment — COMP50074 Professional Practice and Project Management, second year, second semester. Group project by Isuru, Fazl, Thariq and Ahmed.